

Классические методы параллельного программирования

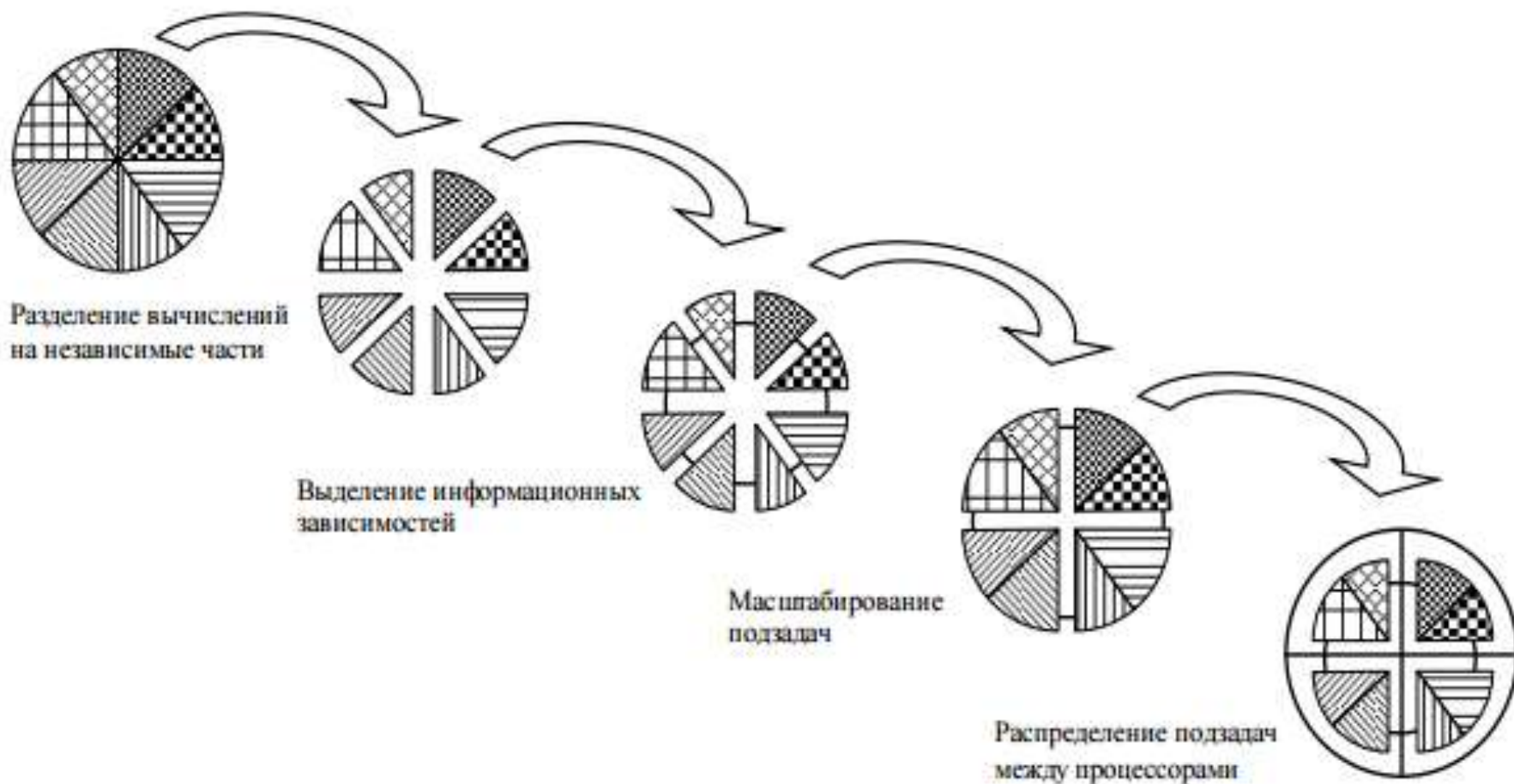
Общая схема разработки параллельных методов

- Выполнить анализ имеющихся вычислительных схем и осуществить их разделение (**декомпозицию**) на части (подзадачи), которые могут быть реализованы в значительной степени независимо друг от друга.
- Выделить для сформированного набора подзадач информационные взаимодействия, которые должны осуществляться в ходе решения исходной поставленной задачи.
- Определить необходимую (или доступную) для решения задачи вычислительную систему и выполнить распределение имеющего набора подзадач между процессорами системы.

- Объем вычислений для каждого используемого процессора должен быть примерно одинаков – это позволит обеспечить равномерную вычислительную загрузку (балансировку) процессоров.
- Распределение подзадач между процессорами должно быть выполнено таким образом, чтобы наличие информационных связей (коммуникационных взаимодействий) между подзадачами было минимальным

- Часто выполняемым дополнительным действием в приведенной выше схеме проектирования является корректировка состава сформированного множества задач после определения имеющегося количества процессоров – подзадачи могут быть укрупнены (**агрегированы**) при наличии малого числа процессоров или, наоборот, **детализированы** в противном случае.
- В целом, данные действия могут быть определены как **масштабирование** разрабатываемого алгоритма и выделены в качестве отдельного этапа проектирования параллельных вычислений.

Общая схема разработки параллельных алгоритмов



Этапы разработки параллельных алгоритмов

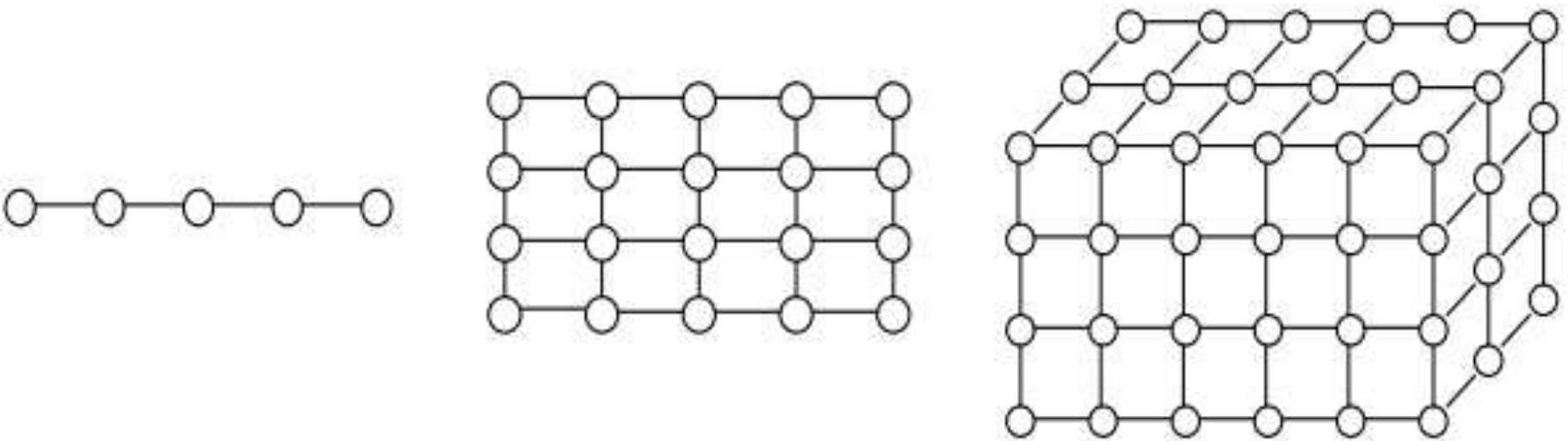
1. Разделение вычислений на независимые части
2. Выделение информационных зависимостей
3. Масштабирование набора подзадач
4. Распределение подзадач между вычислительными элементами

1. Разделение вычислений на независимые части

- Для большого класса задач вычисления сводятся к выполнению однотипной обработки большого набора данных – к такому виду задач относятся, например, матричные вычисления, численные методы решения уравнений в частных производных и др. В этом случае говорят, что существует **параллелизм по данным**, и выделение подзадач сводится к разделению имеющихся данных.

- Для другой части задач вычисления могут состоять в выполнении разных операций над одним и тем же набором данных – в этом случае говорят о существовании **функционального параллелизма** (в качестве примеров можно привести задачи обработки последовательности запросов к информационным базам данных, вычисления с одновременным применением разных алгоритмов расчета и т. п.). Очень часто функциональная декомпозиция может быть использована для организации конвейерной обработки данных (так, например, при выполнении каких-либо преобразований данных вычисления могут быть сведены к функциональной последовательности ввода, обработки и сохранения данных).

Регулярные одно-, двух- и трехмерные структуры базовых подзадач после декомпозиции данных



Для оценки корректности этапа разделения вычислений на независимые части можно воспользоваться контрольным списком вопросов:

- Выполненная декомпозиция не увеличивает объем вычислений и необходимый объем памяти?
- Возможна ли при выбранном способе декомпозиции равномерная загрузка всех имеющихся вычислительных элементов компьютерной системы?
- Достаточно ли выделенных частей процесса вычислений для эффективной загрузки имеющихся вычислительных элементов (с учетом возможности увеличения их количества)?

2. Выделение информационных зависимостей

- **Локальные и глобальные схемы** информационного взаимодействия – для локальных схем в каждый момент времени информационная зависимость существует только между небольшим числом подзадач (располагаемых, как правило, на соседних вычислительных элементах), для глобальных зависимостей информационное взаимодействие имеет место для всех подзадач.
- **Структурные и произвольные способы взаимодействия** – для структурных способов организация взаимодействий приводит к формированию некоторых стандартных схем коммуникации (например, в виде кольца, прямоугольной решетки и т. д.), для произвольных структур взаимодействия схема информационных зависимостей не носит характера какой-либо

- **Статические или динамические схемы** информационной зависимости – для статических схем моменты и участники информационного взаимодействия фиксируются на этапах проектирования и разработки параллельных программ, для динамического варианта взаимодействия структура зависимостей определяется в ходе выполняемых вычислений.

Список контрольных вопросов

- Соответствует ли вычислительная сложность подзадач интенсивности их информационных взаимодействий?
- Является ли одинаковой интенсивность информационных взаимодействий для разных подзадач?
- Является ли схема информационного взаимодействия локальной?
- Не препятствует ли выявленная информационная зависимость параллельному решению подзадач?

3. Масштабирование набора подзадач

- Масштабирование разработанной вычислительной схемы параллельных вычислений проводится в случае, когда количество имеющихся подзадач отличается от числа планируемых к использованию вычислительных элементов.

- Для сокращения количества подзадач необходимо выполнить укрупнение (агрегацию) вычислений.
- При недостаточном количестве имеющегося набора подзадач для загрузки всех доступных к использованию вычислительных элементов необходимо выполнить детализацию (декомпозицию) вычислений.
- Выполнение этапа масштабирования вычислений должно свестись, в конечном итоге, к разработке правил агрегации и декомпозиции подзадач, которые должны параметрически зависеть от числа вычислительных элементов, применяемых для вычислений.

Список контрольных вопросов

- Не ухудшится ли локальность вычислений после масштабирования имеющегося набора подзадач?
- Имеют ли подзадачи после масштабирования одинаковую вычислительную и коммуникационную сложность?
- Соответствует ли количество задач числу имеющихся вычислительных элементов?
- Зависят ли параметрически правила масштабирования от количества вычислительных элементов?

4. Распределение подзадач между вычислительными элементами

- Распределение подзадач между вычислительными элементами является завершающим этапом разработки параллельного метода.
- Управление распределением нагрузки для процессоров возможно только для вычислительных систем с распределенной памятью.
- Для мультипроцессоров (систем с общей памятью) распределение нагрузки обычно выполняется операционной системой автоматически.

- Основной показатель успешности выполнения данного этапа – эффективность использования вычислительных элементов, определяемая как относительная доля времени, в течение которого вычислительные элементы использовались для вычислений, связанных с решением исходной задачи.

- Краткая характеристика широко используемого способа динамического управления распределением вычислительной нагрузки, обычно именуемая схемой **«менеджер–исполнитель»** (manager–worker scheme).
- При использовании данного подхода предполагается, что подзадачи могут возникать и завершаться в ходе вычислений, при этом информационные взаимодействия между подзадачами либо полностью отсутствует, либо минимальны.
- В соответствии с рассматриваемой схемой для управления распределением нагрузки в системе выделяется отдельный вычислительный элемент-менеджер, которому доступна информация о всех имеющихся подзадачах.

- Остальные вычислительные элементы системы являются исполнителями, которые для получения вычислительной нагрузки обращаются к менеджеру.
- Порождаемые в ходе вычислений новые подзадачи передаются обратно менеджеру и могут быть получены для решения при последующих обращениях исполнителей.
- Завершение вычислений происходит в момент, когда исполнители завершили решение всех переданных им подзадач, а менеджер не имеет каких-либо других вычислительных работ для выполнения.

Список контрольных вопросов

- Не приводит ли распределение нескольких задач на один процессор к росту дополнительных вычислительных затрат?
- Существует ли необходимость динамической балансировки вычислений?
- Не является ли вычислительный элемент-менеджер «узким» местом при использовании схемы «менеджер—исполнитель»?